From Monolith to Multi-Pillar:

Scaling MCP/RAG Code Intelligence Across the OMD Portfolio

A Six-Month Retrospective and the Architectural Inflection Point

NOAA Office of Modeling Development — Configuration Management Discovery Year 1

Terry McGuinness
Computing & Advanced Technologies (CAT) Unit
Environmental Modeling Center, NCEP/NWS/NOAA
terry.mcguinness@noaa.gov

May 22, 2026

Abstract

Over six months we built, deployed, and validated a 52-tool MCP/RAG server (`agentcore-mcp-rag`) that puts AI-assisted code intelligence on top of NOAA's Global Forecast System workflow. The system ingests ∼149 000 graph nodes (Neptune) and ∼199 000 documents (OpenSearch) drawn from the `develop` branch of `NOAA-EMC/global-workflow` and the surrounding ecosystem (EE2 standards, J-Job documentation, ingested community summaries, external documentation crawls). The server is deployed as an AWS Bedrock AgentCore Runtime and accessed by developers through a CAC-authenticated bastion path from their GFE laptops. Every objective on the original deployment scorecard has been met: HEALTHY 4/4 on every component, 7/9 on the per-tool functional smoke suite, sub-200 ms latency on representative queries, and live 100% similarity on domain-specific searches such as MPAS Voronoi mesh content that we explicitly rebuilt from a path-prefix crawler bug last week.

This paper argues that hitting that scorecard is not the end of the story — it is the moment the next-level architectural problem becomes visible. The deep dive on `dev/sfs` (the freshly connected Seasonal Forecast System branch) revealed that `NOAA-EMC/global-workflow.git` is not one system but **six**: `develop` plus five active `dev/*` pillars (`dev/gfs.v17`, `dev/gfs.v16`, `dev/gcafs.v1`, `dev/jedi-gfs`, `dev/sfs`), each representing a distinct intellectual program with its own user community, divergence trajectory, and lifecycle. The current monolithic ingestion of `develop` is silently misleading users working on the other five pillars.

We present the multi-pillar architecture that resolves this — a catalog-driven tenant model with branch-scoped data isolation, content- addressed dedupe, shared-component inheritance, and lifecycle-aware attribution — and we identify `dev/sfs` as the natural pilot target. The paper is structured as a one-stop briefing for OMD management: the journey, the inflection point, the architecture, the risk-managed rollout, and the one-week-actionable next step.

1 The Six-Month Journey

1.1 Where we started (December 2025)

The original premise was straightforward: take the GFS workflow code, build a Neo4j graph from the AST + import + invocation relationships, build a ChromaDB vector store from the documentation and code-with- context corpora, and expose a Model Context Protocol (MCP) server with tools for code analysis, semantic search, EE2 compliance checks, and operational guidance. A

developer using the Kiro IDE would get GraphRAG-quality answers about j-jobs, ex-scripts, ush helpers, and parm configurations without leaving their editor.

Phase 48 ported that vision to AWS-native services. Neo4j became Neptune. ChromaDB became OpenSearch. Sentence-Transformers `all-mpnet-base-v2` became Bedrock Titan Embed Text v2 at 1024 dimensions. The Node.js runtime (still in production for backward compatibility) became a Python port that runs as an ARM64 container on Bedrock AgentCore Runtime. Every piece of the data plane lives inside a private VPC; there is no internet-facing service surface. CAC/PIV authentication carries the identity from the GFE laptop, through an NIH bastion, into the EC2 development host, and through to AgentCore via IAM SigV4.

1.2 Where we are (May 22, 2026)

The system that exists today:

- **52 tools across 9 modules**: workflow info, code analysis, semantic search, EE2 compliance, operational guidance, GraphRAG, GitHub integration, SDD workflow tracking, and built-in utilities.
- **Data plane**: Neptune (148 976 nodes, 2.82M relationships) + OpenSearch (15 indices, 198 832 documents spanning the workflow itself, J-Job headers, code-with-context, EE2 standards, and community summaries).
- **Live AgentCore Runtime**: `mdc_mcp_rag_server_python-v5K2F8BGrN`, version 16, image `python-titan-v5`, status READY.
- **Health**: HEALTHY 4/4 on the synthetic check; 7/9 pass on the new functional smoke suite (with two known fails being pre-existing container-state gaps now under remediation).

1.3 The week-of-arrival path

The last six weeks alone delivered: Phase C-3 Bedrock IAM enablement; the Python cutover (now the active MCP target); Bedrock-native embedding swap to Titan 1024; manifest gap closure (12 pending URL sources, including newly added MPAS, CATChem, CECE, CDEPS, Land-DA, uwtools, UFS-SRW, GSI, HAFS); the MPAS path-prefix crawler bugfix that turned `doc_count: 0` into `doc_count: 439` real Voronoi-mesh documents; functional smoke tests that catch "tool registered but data layer broken" silently-passing failures; governance changes that prevent recurrence (Spec-First gate, git operation policy, edit-time hooks); and an end-to-end CAC connectivity diagram that lets a new developer land on the system in two hours.

The story is not "we are done." The story is "we did exactly what we set out to do, and the act of doing it surfaced the next problem."

2 The Inflection Point: `dev/sfs` Forces the Multi-Pillar Question

2.1 What the deep dive showed

On May 22, 2026 a new submodule `global-workflow_dev-sfs` was added to track the Seasonal Forecast System branch. We performed a deep dive on it, expecting an incremental feature branch. What we found, summarized in Table **??**, is that the `NOAA-EMC/global-workflow.git` repository hosts *six living trajectories* that all share the same parent commit history but have meaningfully different code, intent, user communities, and lifecycle states.

Table 1: The six development pillars inside one repository (state on 2026-05-22)

Pillar	Purpose	Ahead	Behind	Diff (files)
`develop`	Canonical operational baseline	—	—	—
`dev/gfs.v17`	Next-generation coupled GFS (operational target)	71	163	894
`dev/gfs.v16`	Current operational v16 (parallel evolution)	855	2 053	2 287
`dev/gcafs.v1`	Global Chemistry & Aerosol Forecast v1	33	115	667
`dev/jedi-gfs`	JEDI-based DA experimentation (5 mo stale)	0	179	834
`dev/sfs`	S2S post-processing pipeline (new — in PR review)	284	0	112

The `dev/sfs` branch alone adds three new J-Jobs (`JGLOBAL_ATMOS_POST`, `_OCN_POST`, `_ICE_POST`), six new ush helpers (`process_atmos_{6hrly,daily}.sh`, `process_ocean_{6hrly,daily,monthly}.sh`), three Python ocean-diagnostics scripts implementing CPC operational products (Ocean Heat Content, Tropical Cyclone Heat Potential, depth of 20 °C isotherm), 20 Jinja archive templates parameterized per ensemble member, 11 CI test cases, 7 platform module files, and the workflow generator integration to wire it all into the Rocoto task graph.

None of these artifacts exist on `develop`.

2.2 The configuration-management hazard, made concrete

A user working on `dev/sfs` who asks the AgentCore MCP "what are the inputs to `JGLOBAL_ATMOS_POST`?" receives, today, the answer: "not found." That is not the worst case — it is loud and the user knows to look elsewhere.

The worst case is when a file *exists on both branches with divergent behaviour*. A user working on `dev/gfs.v17` who asks "what does the forecast driver do?" receives a develop-branch answer that is silently wrong. The 894-file divergence guarantees this class of false-positive will grow as v17 development progresses. The question is not *whether* this is a problem — it is, today, on develop alone — but *how soon* the operational community notices that the AI assistant is feeding them stale information.

2.3 Why this is the right time to address it

Three properties of the current moment make this the architecturally right time to act:

1. **The single-tenant system is healthy and proven.** Six months of investment has produced a working baseline. Multi-tenancy can be added *on top of* a working system rather than retrofit into a struggling one.

2. **The smallest possible pilot is available.** `dev/sfs` (112 files / +3 829 lines, fully synced with develop today) is the cheapest possible target on which to validate the approach before scaling to `dev/gfs.v17` (where the operational stakes are highest).

3. **The governance to prevent silent regressions is now in place.** The Spec-First gate, the functional smoke suite, the git operation policy, and the file-edit hooks were all built in the last two weeks specifically to catch the kind of "everything passes but the data layer is wrong" failure that multi-tenancy makes more probable.

3 The Multi-Pillar Architecture

3.1 Core principle: same image, different data view

Figure ?? shows the proposed topology. Every tenant runs the *same* Docker image. The 52-tool surface is identical; the tools have no idea which tenant they are answering for. What differs between tenants is:

- The OpenSearch index prefix (e.g. `gw_sfs_workflow_docs` vs `gw_workflow_docs`).
- The Neptune label prefix (e.g. `GW_SFS_File` vs `GW_File`) since AWS Neptune does not have multi-database semantics.
- The `WORKFLOW_REF` environment variable (e.g. `NOAA-EMC/global-workflow@dev/sfs` vs `...@develop`).
- The `tenant_id` attached to every response for attribution.

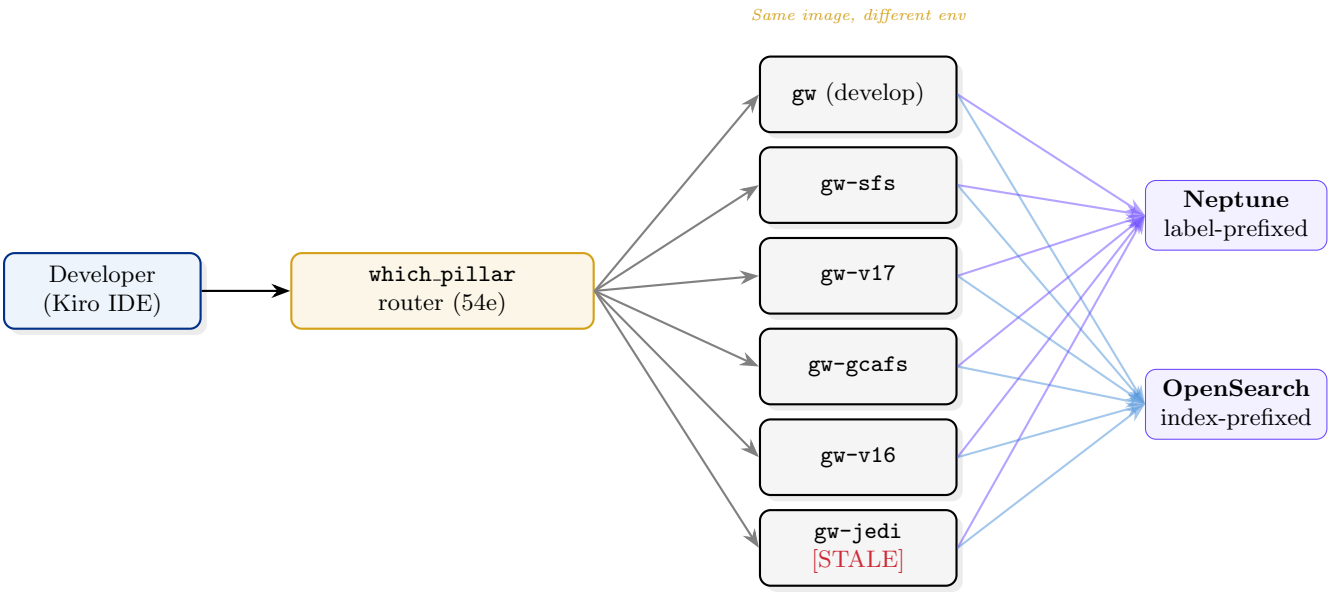

Figure 1: Multi-pillar topology. One image, one router, N tenants, one shared data plane with prefix-based scoping.

3.2 Workstream decomposition

Table 2: The seven workstreams of the OMD Multi-Pillar Initiative

ID	Workstream	What it delivers
54a	Tenant catalog schema	YAML schema: `tenant_id`, repo, branch, `extends:`, lifecycle, staleness, secrets
54b	Per-tenant data isolation	OpenSearch index prefix + Neptune label prefix; 52 tools made tenant-aware
54c	Shared-component inheritance	`extends: [ufs]` with catalog-time vs query-time semantics decided
54d	Branch-scoped tenants + dedupe	Six `gw-*` tenants on the same image; SHA-keyed content dedupe (critical-path)
54e	Routing & attribution	`which_pillar` recommender; `tenant_id` on every response
54f	Headless auth broker (parallel)	`age`/`sops` or Vault; arXiv + Semantic Scholar pilots
54g	Lifecycle & staleness (NEW)	Auto-deprecation on branch merge; `[STALE]` marker at threshold

3.3 Pilot rollout sequence

The Initiative is decomposed into a risk-managed rollout that validates the cheapest possible pilot before committing to the operational prize:

1. `omd-tenants-1-foundation` (54a + 54b minimum). Existing AgentCore Runtime becomes a single tenant `gw`; nothing changes for users but plumbing is in place.
2. `omd-tenants-2-sfs-pilot` (54d). Stand up `gw-sfs` against `dev/sfs`, ingest only the 112-file diff, run the branch-isolation probe, prove dedupe works.
3. `gw-v17-tenant`. Repeat for `dev/gfs.v17` — the operational prize. The v17 community starts getting AI-assisted code analysis that is correct.
4. Remaining pillars (`gw-gcafs`, `gw-v16`, `gw-jedi`) in order of community urgency.
5. `extends:` (54c) + UFS tenant when 2+ pillars share UFS.
6. Routing + lifecycle (54e + 54g) when 3+ tenants exist.
7. Auth broker (54f) parallel to all of the above.

4 Cost & Risk

4.1 Storage / cost projections

Naive 6× ingestion would 6× the data plane cost. Content-addressed dedupe (workstream 54d) is what makes the model economically viable. The merge-base structure of the six pillars is favourable: `dev/sfs` shares 99.6% of files with develop; `dev/gfs.v17` shares 90.5%; `dev/gcafs.v1` shares 96%. Only the divergent slice needs new vectors and new graph nodes; the rest is referenced from the develop tenant via SHA-keyed dedupe.

Table 3: Approximate ingestion cost per pillar (with dedupe)

Pillar	Diff size	Dedupe basis	Net new $/month
`gw-sfs`	112 files	vs develop (99.6% shared)	~$2
`gw-v17`	894 files	vs develop (90.5% shared)	~$15
`gw-gcafs`	667 files	vs develop (96% shared)	~$8
`gw-v16`	2287 files	vs develop (76% shared)	~$30
`gw-jedi`	834 files	vs develop	~$12
Total above develop baseline			**~$67/month**

This is small compared to the existing baseline (existing Neptune + OpenSearch are sunk; the marginal Bedrock embedding cost is small).

4.2 Top risks

Three risks dominate the rollout:

1. **Content-addressed dedupe correctness (R3).** If dedupe is buggy, two tenants can return each other's data. This is the worst class of failure in a multi-tenant system. Mitigation: a dedicated parity probe in 54d that fires queries known to have different answers per tenant and asserts they do.

2. **Tenant routing UX (R4).** If users have to learn six tenant names, adoption suffers. The `which_pillar` recommender (54e) is on the critical path, not nice-to-have.

3. **Per-tenant runtime cost vs routing complexity (R7).** One AgentCore Runtime per tenant is clean but expensive. One Runtime serving all tenants with header-based routing is cheap but adds a routing layer to operate. Decision: spike both in 54b.

5 What This Means for OMD Management

5.1 Six-month story arc, in one paragraph

In December 2025 we set out to put AI-assisted code intelligence on top of the Global Forecast System. By May 2026 we had a 52-tool production-grade MCP/RAG server running on AWS Bedrock AgentCore, serving CAC-authenticated developers, with ~199K embedded documents and ~2.9M graph relationships. The act of finishing that work surfaced the next problem: the OMD portfolio is not one program but at least six (and growing), and the monolithic system serves only one of them. The architecture to address that *(multi-pillar tenant model with branch-scoped data isolation and lifecycle-aware attribution)* is well-understood, the pilot target *(`dev/sfs`)* is the smallest possible validation of the approach, and the engineering that prevents silent regressions *(spec-first gate, functional smoke tests, edit-time hooks)* is already in place.

5.2 Why this paper is the publication

Six months of work culminate in an architectural inflection. The publishable insight is not "we built an MCP server" — many groups have done that — it is the journey from *single-program monolith* to *recognition that scientific computing organizations maintain portfolios of intellectually distinct development trajectories that look like branches of one repository but are functionally*

separate systems. Configuration-management hazard, content-addressed dedupe, tenant lifecycle, branch-isolation probes, and auto-deprecation on merge are concepts that generalize beyond NOAA — any organization maintaining N branches of one operational system has the same problem.

5.3 The one-week-actionable next step

This paper accompanies the Phase 54 SDD Initiative (`Phase-54-OMD-Multi-Program-MCP-Initiative` on the `global-workflow.wiki`) and a companion Kiro spec `omd-tenants-1-foundation` on `develop_aws`. The recommended first action: author and approve the spec for workstreams 54a + 54b, with `dev/sfs` as the validation target. The total engineering effort to get from "spec approved" to "branch-isolation probe passing on `gw-sfs`" is estimated at one engineering sprint (two weeks).

6 References & Companion Artifacts

- **Wiki Initiative**: `Phase-54-OMD-Multi-Program-MCP-Initiative.md` (refreshed 2026-05-22)
- **Connectivity diagram**: `docs/presentations/kiro_cac_connectivity.pdf`
- **Knowledge base paper**: `docs/presentations/papers/rag_knowledge_base/main.pdf`
- **Functional smoke tests**: `.kiro/specs/functional-smoke-tests/`
- **Governance**: `.kiro/steering/02-development-workflow.md`, `07-feature-branch-spec-workflow.md`, `08-git-operation-policy.md`
- **Pilot target submodule**: `supported_repos/global-workflow_dev-sfs` (branch `dev/sfs`, HEAD `4f9ff41f4`)